DeepGroundRepair: A Tool for API-Grounded Generation and Shape-Aware Repair of Deep Learning Programs

Xiangyue Ma, Zhen Wang, Jun Li, Haotian Li, and Zheng Zheng
Beihang University, Beijing, China

***Abstract*—Large language models (LLMs) can generate deep learning (DL) programs from natural-language requirements, but the generated programs may contain API-level syntactic issues or fail at runtime because of tensor-shape mismatches. We present the tool realization of DeepGroundRepair, which integrates API-grounded generation, failure localization, and shape-aware repair into an executable workflow. It grounds draft generation in task-relevant API knowledge, localizes upstream API configurations behind tensor-shape failures, and uses runtime tensor shapes to guide parameter repair. Re-execution validates the resulting program. DeepGroundRepair supports TensorFlow, PyTorch, and PaddlePaddle and is intended for DL developers and software engineering researchers working with LLM-generated DL code. The artifact is available at https://doi.org/10.5281/zenodo.22727085; the demonstration video link will be added in the camera-ready version.**

***Index Terms*—deep learning programs, large language models, code generation, failure localization, program repair**

I. Introduction

Deep learning (DL) programs must satisfy library-specific API usage constraints and tensor-shape dependencies. LLM-generated DL programs may therefore contain nonexistent or misused APIs and may also fail at runtime because of tensor-shape mismatches. Such failures are difficult to diagnose because the operation reporting a mismatch can be downstream from the API configuration that introduced the incompatible shape.

A developer may consequently need to move repeatedly between generation, execution, diagnosis, repair, and re-execution. Retrieval-augmented generation can provide external knowledge during generation [3], while execution-feedback methods can refine failed programs [4]. DeepGroundRepair integrates these activities into a DL-specific generation–localization–repair workflow.

DeepGroundRepair follows a *Generate* $\rightarrow$ *Localize* $\rightarrow$ *Repair* workflow. Given a natural-language requirement, a target DL library, and an LLM configuration, the tool produces retrieved API information, a draft program, a failure diagnosis with runtime shape context when execution fails, a repaired program, and a validation result. The intended users are DL developers using LLM-generated code and software engineering researchers studying such programs.

This paper presents the tool realization of the previously proposed DeepGroundRepair approach. Rather than restating the underlying algorithms, we focus on how its generation–localization–repair backend is used in practice, demonstrated on a representative tensor-shape failure, and reproduced through the public artifact.

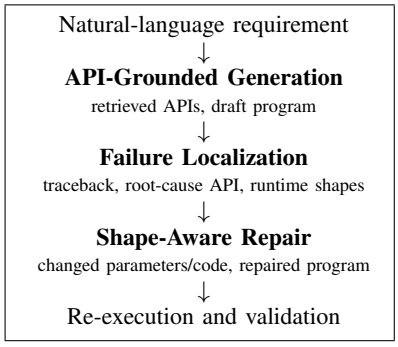


Fig. 1. DeepGroundRepair tool workflow and key user-visible outputs.

II. DeepGroundRepair Tool

DeepGroundRepair is built around a generation–localization–repair backend. API knowledge grounds draft generation, runtime tensor information supports root-cause localization, and the localized context constrains subsequent repair. Figure 1 summarizes this tool-level workflow and the main outputs visible during execution.

API-Grounded Generation. DeepGroundRepair first grounds program generation in task-relevant, library-specific API knowledge. It obtains candidate APIs from the task requirement, verifies their existence, and retrieves concise API information before prompting the LLM to generate a draft DL program. This step aims to reduce API hallucination and parameter misuse and produces the draft program used by the subsequent stages.

Failure Localization. If the draft program fails because of a tensor-shape mismatch, DeepGroundRepair executes the program and combines the traceback with runtime tensor information to trace the mismatch from its failure site to an upstream API configuration. The resulting localization identifies the root-cause API, suspicious parameters, and relevant tensor shapes, thereby narrowing the repair space beyond the traceback location alone.

Shape-Aware Repair. DeepGroundRepair uses the localized API context and traced tensor shapes as explicit repair evidence. It generates candidate parameter changes or code edits, checks them against available shape constraints, and inserts

TABLE I
REPRESENTATIVE VALIDATION EVIDENCE FROM THE ASSOCIATED STUDY.

Capability	Representative evidence
API-grounded generation	2.14 fewer API hallucinations and 3.26 fewer parameter misuses per 10 programs
Failure localization	82.5% root-cause API localization accuracy on 120 annotated cases
Shape-aware repair	31/50/38 fixed shape-mismatch cases, versus 6/15/10 without shape guidance (TF/PT/PD)
End-to-end workflow	86.0% average relative improvement in code executability

a validated repair into the program. The repaired program is then re-executed to determine whether the failure has been resolved. Detailed retrieval, localization, and repair algorithms are described in the associated research study [1]; this paper focuses on their tool-level integration and usage.

III. TOOL USAGE AND REPRESENTATIVE DEMONSTRATION

The current artifact supports TensorFlow, PyTorch, and PaddlePaddle and runs with locally deployed open-weight LLMs. Users specify a natural-language task requirement and target DL library, after which DeepGroundRepair runs generation, localization, and repair in sequence. In the current prototype, the workflow is invoked through the implemented stage chain, from generation to tensor-shape debugging and repair. The artifact repository provides the corresponding code and replication materials.

For a representative demonstration, we use a PyTorch tensor-shape mismatch from the associated study. The generated program defines `nn.Linear(in_features=512*7*7, out_features=1024)`, but the tensor reaching this layer is flattened to shape `(1, 401408)`. Execution therefore fails at `self.fc1(x)` with an incompatible matrix-shape error. DeepGroundRepair uses the traceback and runtime shape trace to map the downstream failure site back to the `nn.Linear` constructor and identifies `in_features` as the suspicious parameter.

The repair component then uses the observed input dimension to constrain the modification, replaces the incompatible value with a shape-consistent one, and re-executes the program. The demonstration therefore exposes the complete user-visible path: *requirement* $\rightarrow$ *retrieved APIs and draft program* $\rightarrow$ *runtime failure* $\rightarrow$ *root-cause diagnosis* $\rightarrow$ *repair difference* $\rightarrow$ *validation result*. A compact before/after code view can be used to present this fixed demonstration without repeating the underlying algorithms.

IV. VALIDATION AND REPRODUCIBILITY

The associated research study evaluates DeepGroundRepair on 100 model-level DL programming tasks using six open-weight LLMs across TensorFlow, PyTorch, and PaddlePaddle [1]. Table I summarizes only the evidence most directly related to the tool capabilities.

The public artifact provides the implementation and replication materials [2]. For the representative demonstration, the final version of this section will report the fixed input, model and target framework, API-index requirements, software and hardware environment, actual invocation command, expected outputs, and approximate runtime. The demonstration video will use the same fixed case so that the observed localization and repair can be reproduced.

DeepGroundRepair currently focuses on API-level syntactic issues and tensor-shape mismatches. Successful re-execution establishes runtime executability rather than complete semantic correctness, and the current workflow does not primarily target dtype, device, training-time, or deployment failures.

V. CONCLUSION

This tool paper presents the executable realization of DeepGroundRepair for LLM-generated DL programs. Starting from a natural-language requirement, the tool generates an API-grounded draft, diagnoses upstream causes of tensor-shape failures, produces a shape-aware repair, and validates the repaired program through re-execution. The released artifact and representative demonstration are intended to make this workflow directly usable and reproducible.

REFERENCES

[1] X. Ma, Z. Wang, X. Lu, X. Du, C. Li, J. Li, H. Li, and Z. Zheng, "DeepGroundRepair: From API-Grounded Generation to Shape-Aware Repair for Deep Learning Programs," manuscript, 2026.

[2] X. Ma, Z. Wang, X. Lu, X. Du, C. Li, J. Li, H. Li, and Z. Zheng, "Artifacts for DeepGroundRepair," Zenodo, 2026. [Online]. Available: https://doi.org/10.5281/zenodo.22727085

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020.

[4] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is Self-Repair a Silver Bullet for Code Generation?" in *Proc. International Conference on Learning Representations*, 2024.